

Déployez un modèle de Machine Learning

Rafael CEREZO MARTIN

Février 2026

Objectifs du projet

Département demandeur : Ressources Humaines (RH)

Problématique : Taux de démission (turnover) anormalement élevé

Objectif principal : Déployer un Proof of Concept (POC) opérationnel pour rendre les modèles de Machine Learning accessibles via une API performante et sécurisée.

Standard Industriel

- **Gestion du Cycle de Vie avec MLflow** : Suivi rigoureux des expérimentations, versionnage des modèles et registre centralisé pour garantir la reproductibilité.
- **Service de Modèle avec BentoML** : Packaging professionnel pour un déploiement optimisé, scalable et prêt pour la production.
- **Traçabilité avec PostgreSQL** : Archivage systématique des logs d'inférence (entrées/sorties) pour l'audit et le monitoring de la performance.

Exigences de Fiabilité

- **Architecture logicielle** : API construite avec FastAPI et Pydantic pour une validation stricte des données.
- **Assurance Qualité** : Suite exhaustive de tests automatisés (Pytest) couvrant la sécurité, la logique métier et la résilience.

1 Diapo
1 Capture des Besoins

1 Diapo
2 Conception Préliminaire

8 Diapos
3 Conception Détaillée La Phase d'Apprentissage

8 Diapos
4 Conception Détaillée La Phase d'Inférence

2 Diapos
5 CI/CD

1 Capture des Besoins

ÉTAPE 1 :

1. Etude Préliminaire

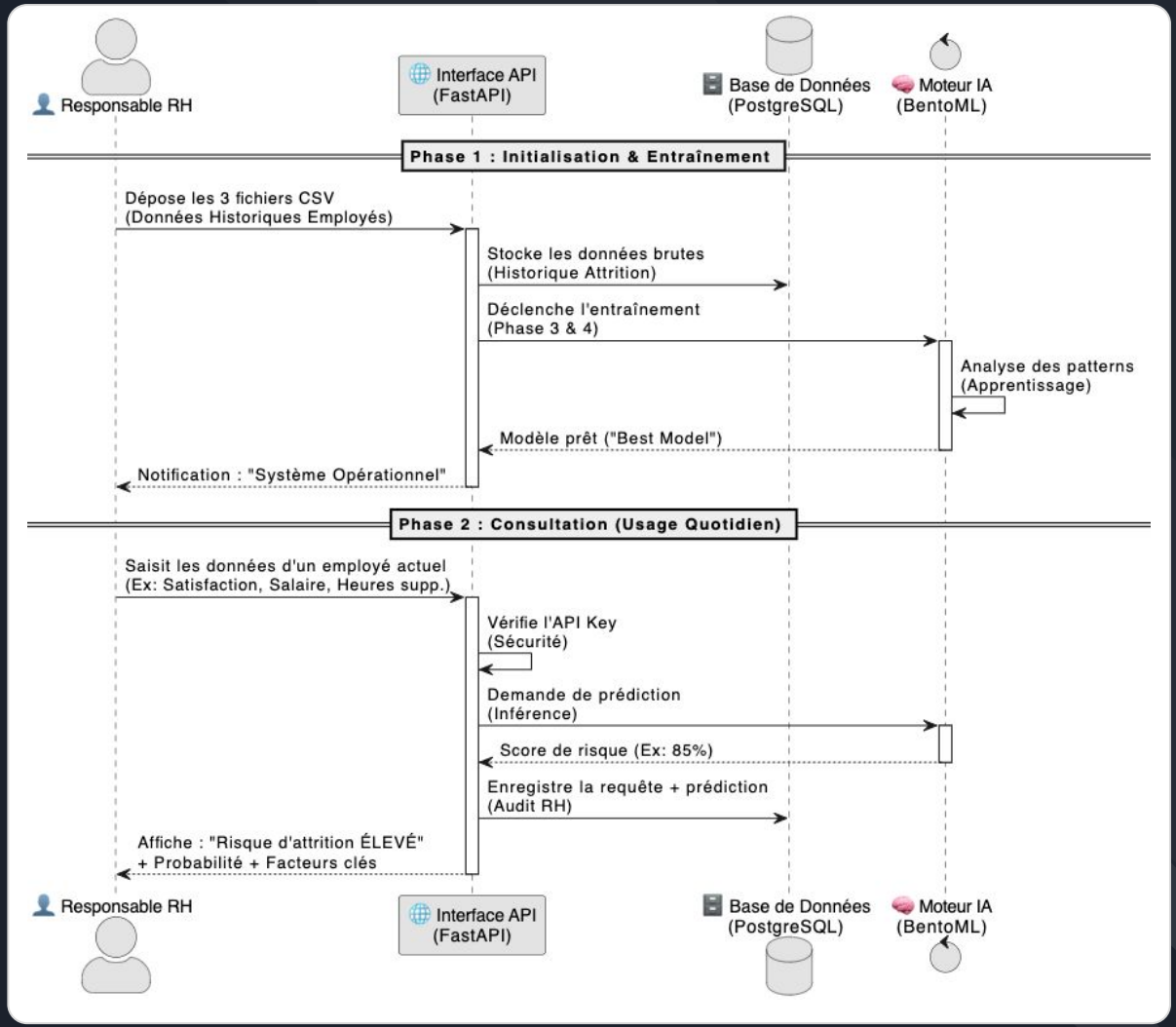
2. Capture des besoins fonctionnels

La Phase d'Apprentissage (Le Passé) : "Le Responsable RH, nous fourni les archives (les 3 CSV). Notre système étudié pourquoi vos anciens employés sont partis. C'est la création de notre **mémoire organisationnelle**."

La Phase d'Inférence (Le Présent/Futur) : "Le Responsable RH quand il a un doute sur un employée, interroge l'API. Rapidement, le système compare le profil actuel à sa mémoire et alerte si un départ est probable."

La Traçabilité (Le Registre) : "Chaque consultation est stockée dans notre base PostgreSQL. Cela nous permet de justifier les plans d'action sociale auprès de la direction."

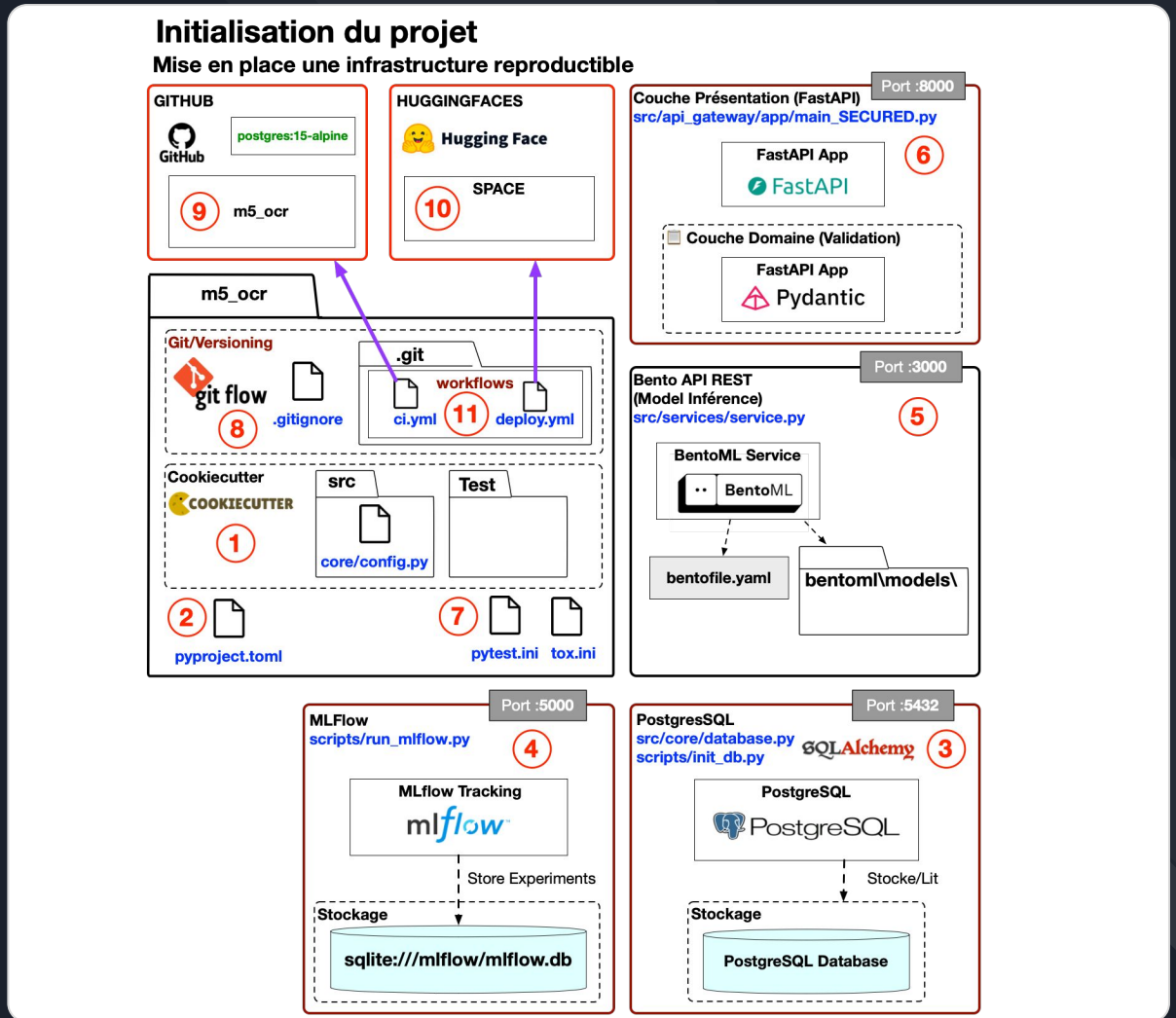
3. Capture des besoins techniques.



2 Conception Préliminaire

ÉTAPE 1 : Initialisation du projet

1. Création de la structure
2. Configuration des libraires
3. Création de l'interface avec la BD
4. Activation du service de tracking MLFlow
5. Configuration du service BentoML
6. Configuration de service FastAPI
7. Test initiaux de vérification des services.
8. Initialization du repository local de Versioning GIT
9. Création du repository distant sur GitHub.
10. Création de l'espace sur HuggingFaces
11. Configuration du Pipeline CI.CD



3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

- Pipeline phase1_preparation.py
- Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

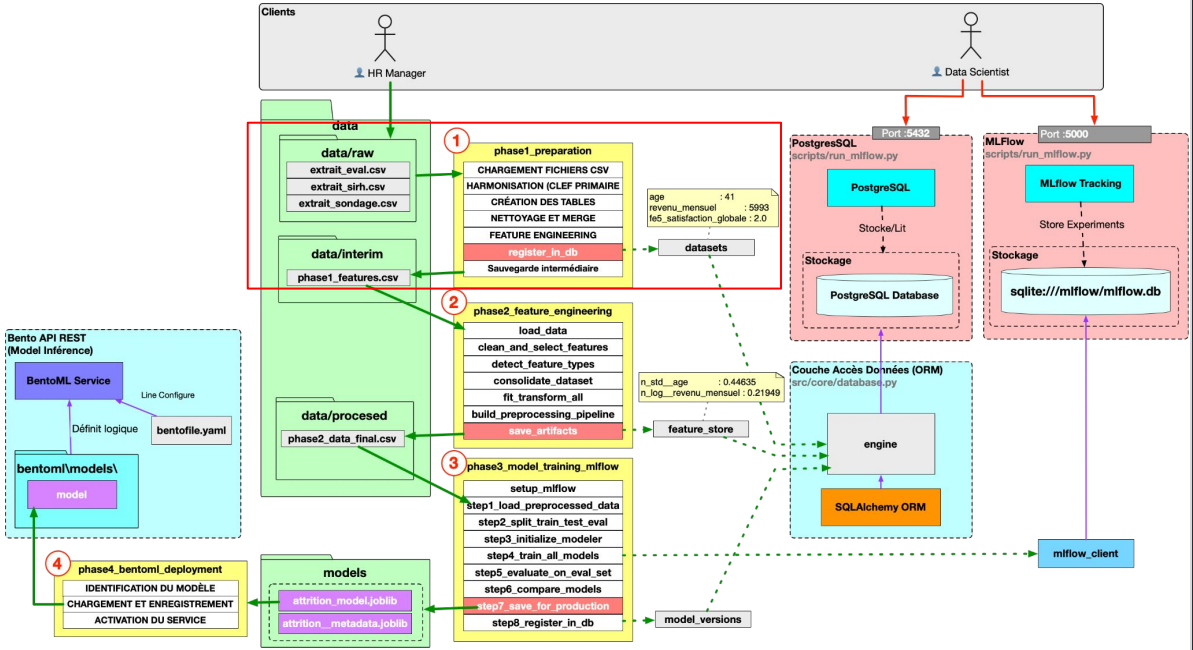
- Consultation de l'entraînement

Phase 4. Configuration du Service BentoML

- Test inference API BentoML (Web)
- Test inference API BentoML (Script)

La Phase d'Apprentissage (Le Passé)

Données brutes >> Feature Engineering >> Training Models >> Model Registry (BentoML)



3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

- Pipeline phase1_preparation.py
- Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

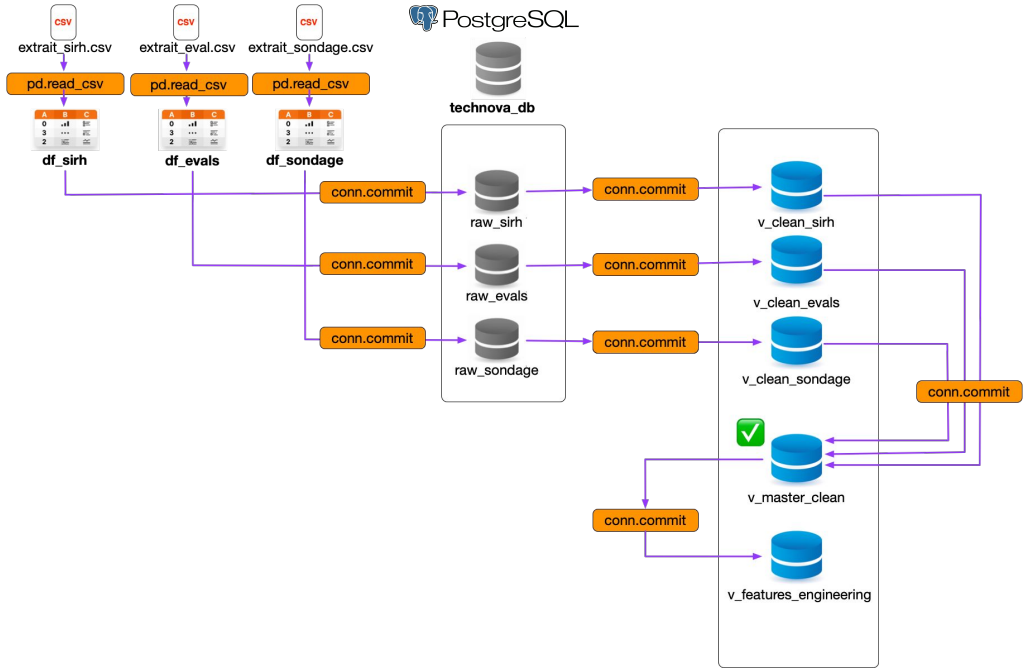
- Consultation de l'entraînement

Phase 4. Configuration du Service BentoML

- Test inference API BentoML (Web)
- Test inference API BentoML (Script)

Phase 1. Préparation des Données (Data Preparation)

Diagramme de Classes UML (ERD - Entity Relationship Diagram)



La Traçabilité (Le Registre)

SELECT * FROM public.datasets

	id [PK] integer	file_path character varying (255)	description text	created_at timestamp without time zone	updated_at timestamp without time zone	version character varying (50)
1	1	data/interim/phase1_features....	Dataset consolidé après nettoyage et feature engineering (Phas...	2026-02-05 13:31:14.124404	2026-02-05 13:31:14.124404	1.0.0

3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

Pipeline phase1_preparation.py

Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

Consultation de l'entraînement

Phase 4. Configuration du Service BentoML

Test inference API BentoML (Web)

Test inference API BentoML (Script)

Diagramme de Classes UML (ERD - Entity Relationship Diagram)



3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

Pipeline phase1_preparation.py

Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

Consultation de l'entraînement

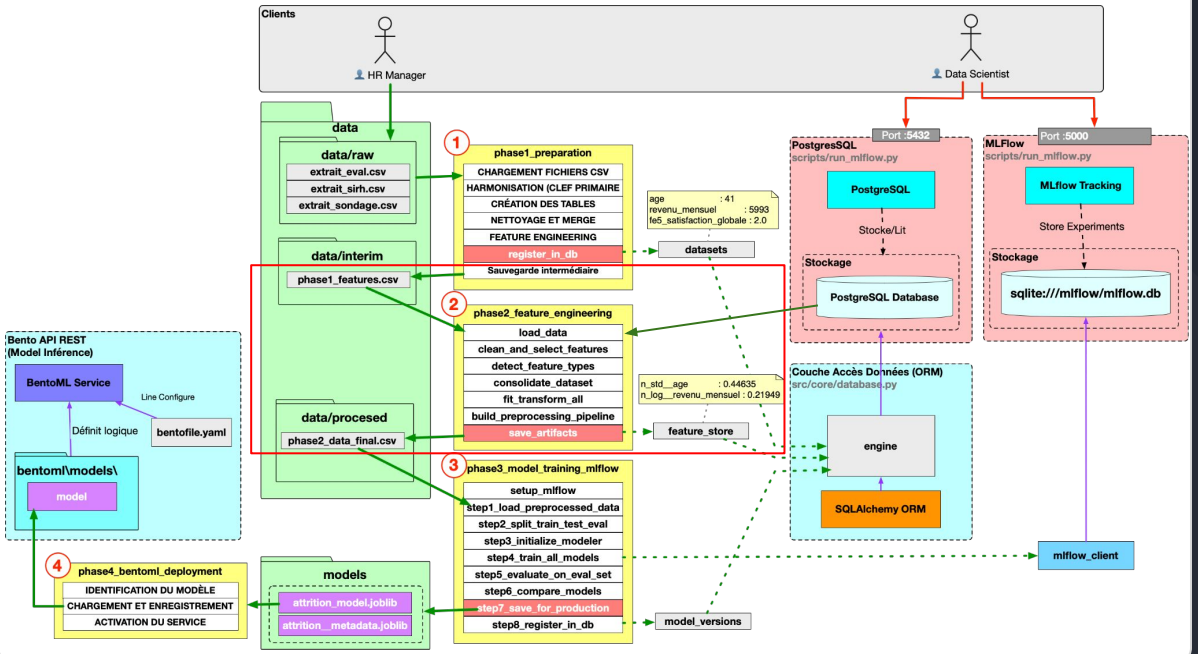
Phase 4. Configuration du Service BentoML

Test inference API BentoML (Web)

Test inference API BentoML (Script)

Phase 2. Preprocessing

Transforme les données brutes en dataset prêt pour l'entraînement.



La Traçabilité (Le Registre)

SELECT * FROM public.feature_store

	id [PK] integer	dataset_id integer	file_path text	row_count integer	feature_count integer	feature_names jsonb
1	1	1	C:\Users\Public\IAE_DELL\pra_dell\m5_ocr\data\processed\phase2_data_final...	1470	47	["c_ohe__genre_M", "c_ohe__statut_marital_Divorcé(e)", "c_oh

3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

Pipeline phase1_preparation.py

Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

Consultation de l'entraînement

Phase 4. Configuration du Service BentoML

Test inference API BentoML (Web)

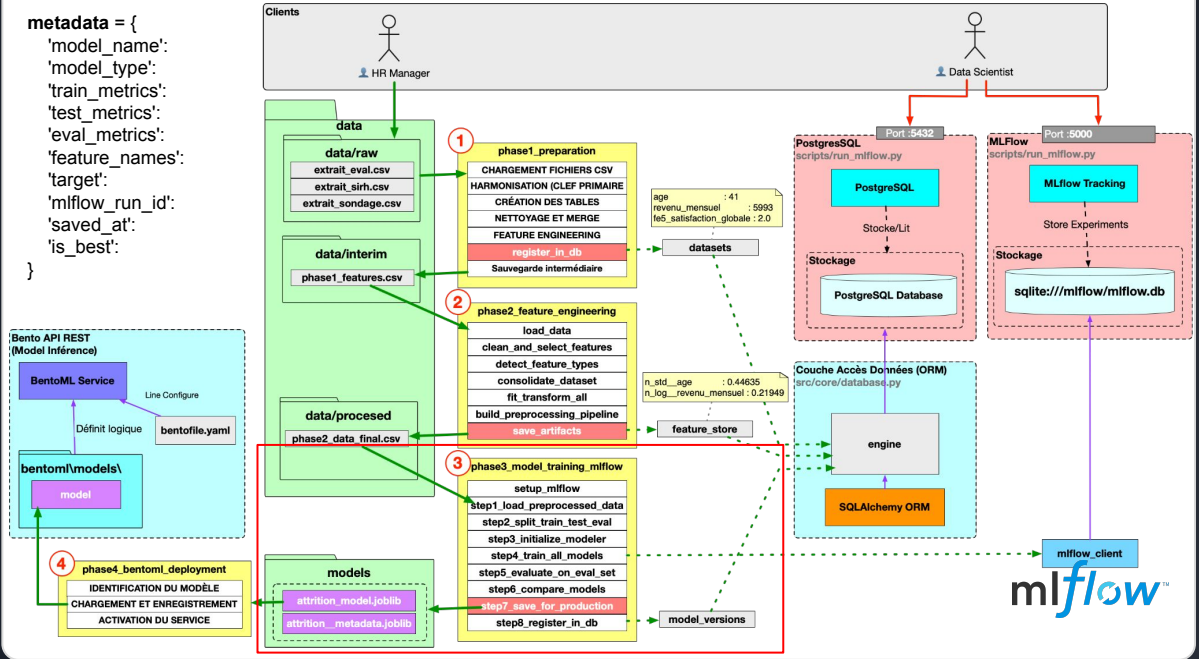
Test inference API BentoML (Script)

Phase 3. Entraînement avec MLflow

Trained Models:
- DummyClassifier
- LogisticRegression
- DecisionTreeClassifier
- RandomForestClassifier
- GradientBoostingClassifier

Logging mlFlow:
1. TAGS
2. PARAMÈTRES
3. MÉTRIQUES
4. MODÈLE
5. ARTIFACTS
6. DATASET

```
metadata = {
    'model_name':
    'model_type':
    'train_metrics':
    'test_metrics':
    'eval_metrics':
    'feature_names':
    'target':
    'mlflow_run_id':
    'saved_at':
    'is_best':
}
```



La Traçabilité (Le Registre)

SELECT * FROM public.model_versions

	id [PK] integer	model_name character varying (255)	version character varying (50)	mlflow_run_id character varying (255)	algorithm character varying (100)	hyperparameters jsonb
1	1	logistic_regression	v_20260205_142708	c48654c3cdd74afabaa5d283bd691a...	LogisticRegression	{"C": 1.0, "tol": 0.0001, "dual": false, "n_jobs": null, "solver": "lbfgs", "penalty":

3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

Pipeline phase1_preparation.py

Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

Consultation de l'entraînement

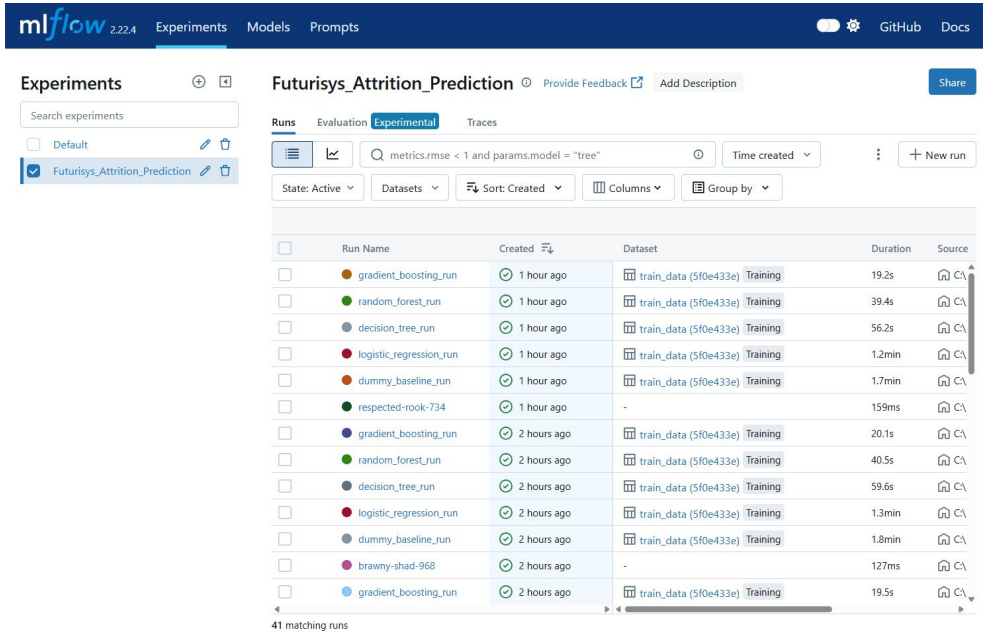
Phase 4. Configuration du Service BentoML

Test inference API BentoML (Web)

Test inference API BentoML (Script)

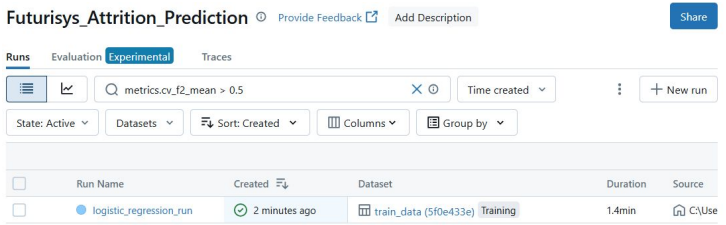
Phase 3. Entraînement avec MLflow

Consultation des entraînement des Modèles . MLflow (Port 5000)



Réalisation de recherches

Identification du meilleur Modèle



3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

Pipeline phase1_preparation.py

Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

Consultation de l'entraînement

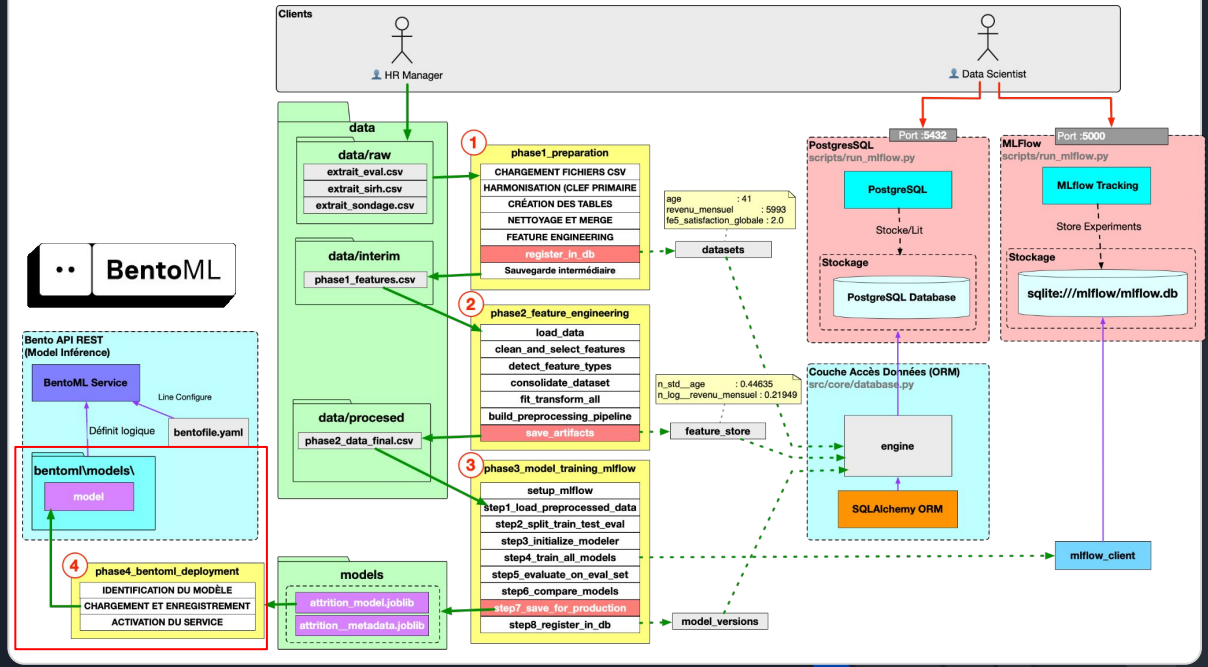
Phase 4. Configuration du Service BentoML

Test inference API BentoML (Web)

Test inference API BentoML (Script)

Phase 4. Configuration du Service BentoML

BentoML transforme le modèle gagnant de MLflow en un service API prêt pour la production.



Model Registry:

```
modele_bento = bentoml.sklearn.save_model(
        "attrition_model",
        objet_modele,
        labels    = {  "phase"      : "production",
                       "model_type" : metadonnees['model_type'] },
        metadata = {  "metrics"    : metadonnees['eval_metrics'],
                       "features"   : metadonnees['feature_names'],
                       "created_at" : datetime.now().isoformat()  }
    )
```

3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

- Pipeline phase1_preparation.py
- Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

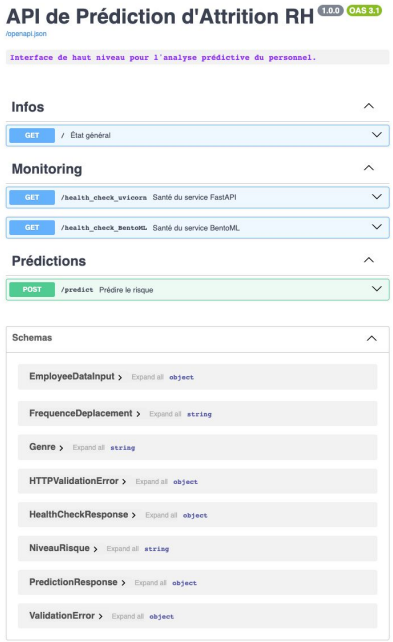
- Consultation de l'entraînement

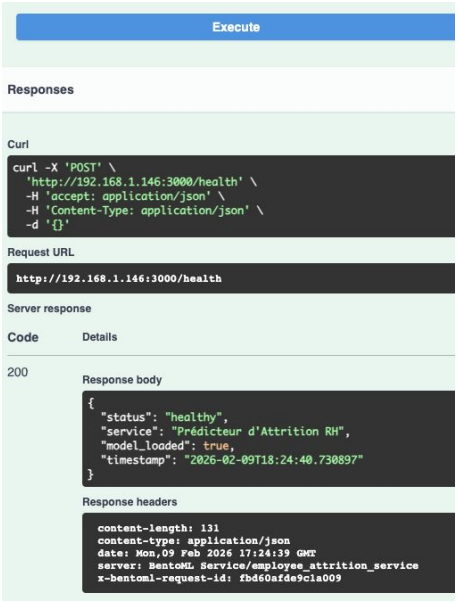
Phase 4. Configuration du Service BentoML

- **Test inference API BentoML (Web)**
- Test inference API BentoML (Script)

Phase 4. Configuration du Service BentoML

Test inference API BentoML (Port 3000)





🌐 Endpoints HTTP

- **POST /predict** → Prédiction
- **GET /health** → Vérification d'état (Health check)
- **GET /docs** → Interface Swagger (Documentation automatique)

3 Conception Détaillée

ÉTAPE 1 : La Phase d'Apprentissage

Phase 1. Préparation des Données (Data Preparation)

Pipeline phase1_preparation.py

Diagramme de Classes UML (ERD - Entity Relationship Diagram)

Phase 2. Preprocessing

Phase 3. Entraînement avec MLflow

Consultation de l'entraînement

Phase 4. Configuration du Service BentoML

Test inference API BentoML (Web)

Test inference API BentoML (Script)

Phase 4. Configuration du Service BentoML

Test inference API BentoML (Script)

```
PS C:\Users\Public\IAE_DELL\pra_dell\m5_ocr> uv run scripts/test_phase4_betoml_api_inference.py
✅ API lista y respondiendo: {'status': 'healthy', 'service': "Prédicteur d'Attrition RH", 'model_loaded': True, 'timest
amp': '2026-02-05T16:13:48.872290'}
🚀 Enviando datos a http://localhost:3000/predict...
------------------------------------------------------------
FEATURE                                                 | VALOR
------------------------------------------------------------
n_std__fe1_ratio_stagnation                             | 0.38
------------------------------------------------------------
✅ Respuesta recibida:
{
    "prediction": 0,
    "probabilite": 0.0839,
    "niveau_risque": "FAIBLE",
    "facteurs_cles": [
        "n_std__fe8_valeur_experience",
        "n_std__fe7_penibilite_trajet",
        "n_std__fe6_risque_overwork"
    ],
    "action_recommandee": "Standard",
    "date_analyse": "2026-02-05T16:13:51.286564"
}
PS C:\Users\Public\IAE_DELL\pra_dell\m5_ocr>
```

ÉTAPE 1 : La Phase d'Apprentissage
Model Registry (Stockage des modèles)

ÉTAPE 2 : La Phase d'Inférence
Service Définition (Logique de l'API)

4 Conception Détaillée

ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

Modélisation SQL

Étape 2 : Le Cœur d'Inférence (BentoML)

Service Bento

Logique de Transformation

Étape 3 : La Passerelle Sécurisée (FastAPI)

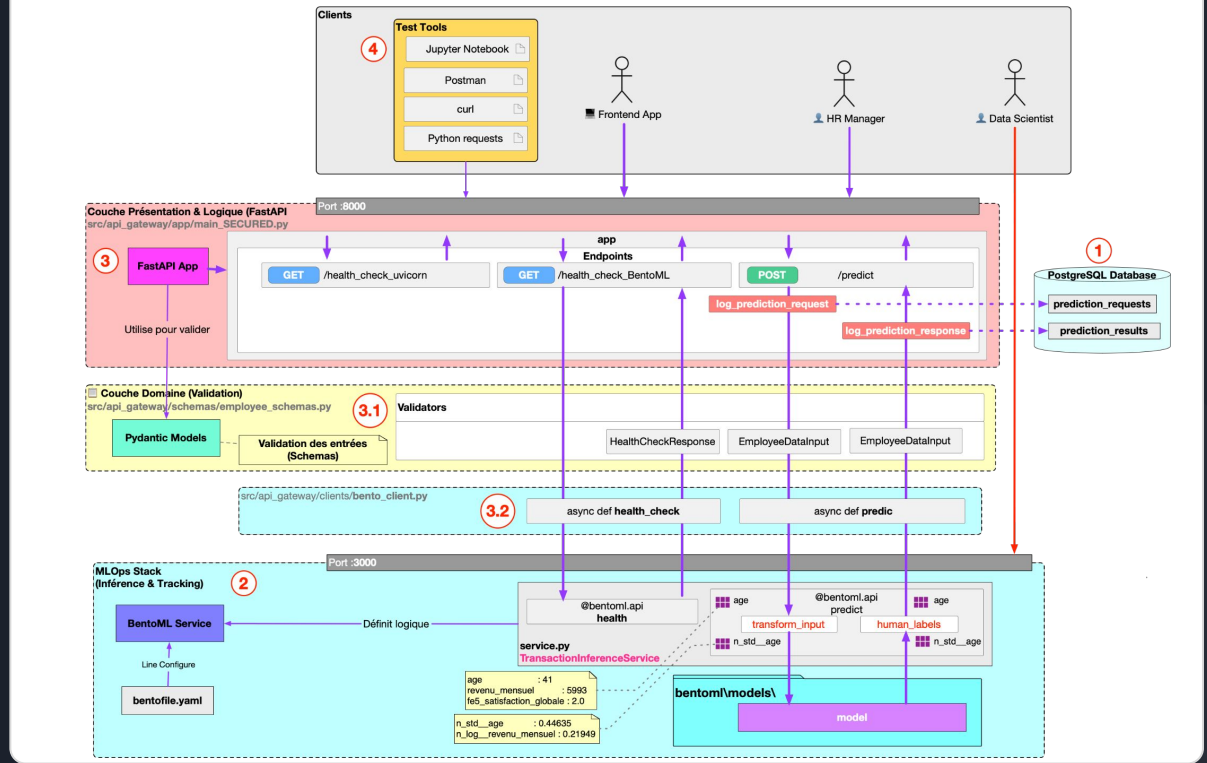
Schémas Pydantic

Client Interne

Étape 4 : Fiabilité et Validation (Tests)

Tests Unitaires

La Phase d'Inférence



La Traçabilité (Le Registre)

SELECT * FROM public.prediction_requests

	id [PK] uuid	created_at timestamp without time zone	model_version_id integer	input_data jsonb
1	c6e3250e-7cd6-4095-85dc-2b327f431...	2026-02-05 14:57:56.900659	1	{"age": 35, "genre": "M", "revenu_mensuel": 3500.0, "frequence_deplacement": "Occasionnel", "heure_s

4 Conception Détaillée

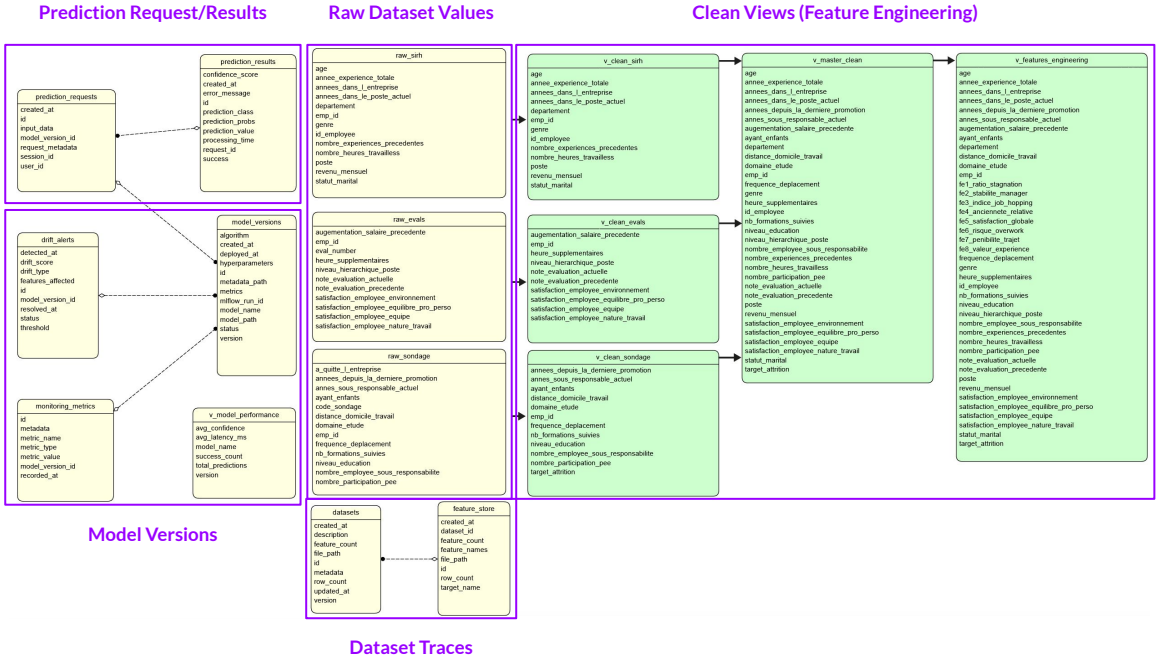
ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

- Modélisation SQL
- Étape 2 : Le Cœur d'Inférence (BentoML)
- Service Bento
- Logique de Transformation
- Étape 3 : La Passerelle Sécurisée (FastAPI)
- Schémas Pydantic
- Client Interne
- Étape 4 : Fiabilité et Validation (Tests)
- Tests Unitaires

Étape 1 : Fondations et Persistance (PostgreSQL)

Diagramme de Classes UML (ERD - Entity Relationship Diagram)



4 Conception Détaillée

ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

Modélisation SQL

Étape 2 : Le Cœur d'Inférence (BentoML)

Service Bento

Logique de Transformation

Étape 3 : La Passerelle Sécurisée (FastAPI)

Schémas Pydantic

Client Interne

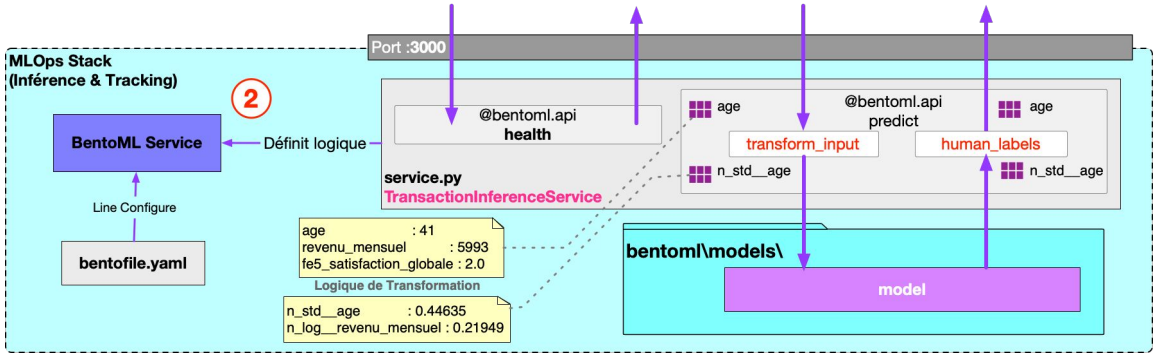
Étape 4 : Fiabilité et Validation (Tests)

Tests de Validation

Étape 2 : Le Cœur d'Inférence (BentoML)

ÉTAPE 1 : La Phase d'Apprentissage
Model Registry (Stockage des modèles)

ÉTAPE 2 : La Phase d'Inférence
Service Définition (Logique de l'API)



REQUEST

```
{
  "employee_data": {
    "age": 35,
    "annees_dans_l_entreprise": 5,
    "annees_dans_le_poste_actuel": 2,
    "fe5_satisfaction_globale": 7.5,
    "frequence_deplacement": "Occasionnel",
    "genre": "M",
    "heure_supplementaires": 10,
    "revenu_mensuel": 3500
  }
}
```

REPONSE

```
{
  "prediction": 1,
  "probabilite": 0.9035,
  "niveau_risque": "CRITIQUE",
  "facteurs_cles": [
    "Heures Supplémentaires",
    "Niveau de Salaire",
    "Âge de l'Employé"
  ],
  "action_recommandee": "Entretien RH urgent",
  "date_analyse": "2026-02-09T16:28:03.159568"
}
```

4 Conception Détaillée

ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

Modélisation SQL

Étape 2 : Le Cœur d'Inférence (BentoML)

Service Bento

Logique de Transformation

Étape 3 : La Passerelle Sécurisée (FastAPI)

Schémas Pydantic

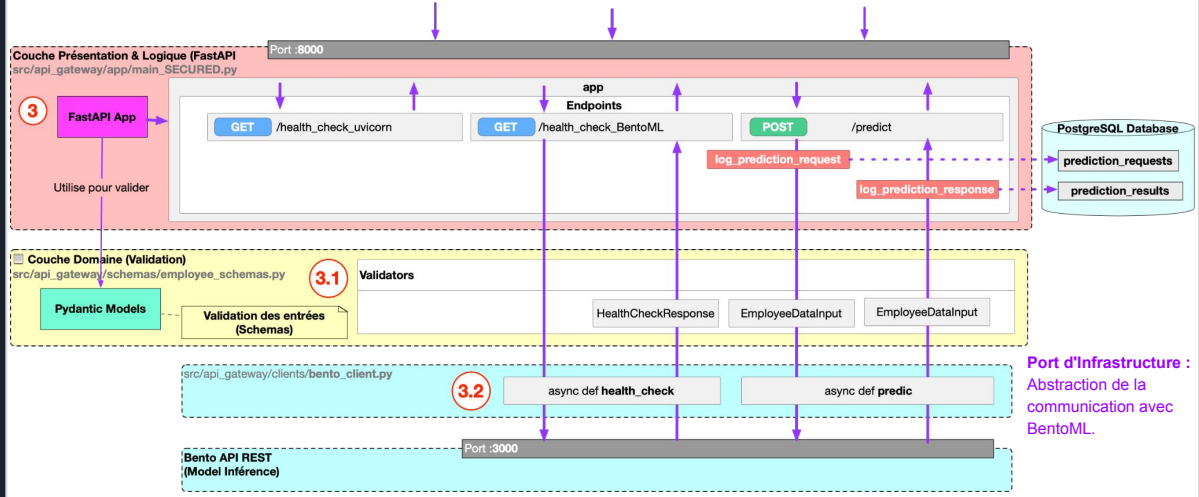
Client Interne

Étape 4 : Fiabilité et Validation (Tests)

Tests de Validation

Étape 3 : La Passerelle Sécurisée (FastAPI)

Le Rôle des Schémas Pydantic dans la Passerelle



Port d'Infrastructure : Abstraction de la communication avec BentoML.

- **Contrat de Données Strict :** Respect du format attendu (types, contraintes, champs obligatoires)
- **Validation Automatique & Typage :** Rejeter instantanément les données erronées
- **Sécurisation de l'Infra :** Protégeant ainsi le service BentoML de crashs potentiels.
- **Auto-Documentation (OpenAPI) :** Les schémas génèrent automatiquement la documentation Swagger, permettant aux développeurs front-end de connaître exactement la structure des objets JSON à envoyer et à recevoir.
- **Sainissement (Sanitization) :** Réponses filtrées et formatées correctement, évitant divulguer des métadonnées sensibles.

REPONSE


```
{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": [
        "body",
        "age"
      ],
      "msg": "Input should be greater than or equal to 18",
      "input": -35,
      "ctx": {
        "ge": 18
      }
    }
  ]
```

4 Conception Détaillée

ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

Modélisation SQL

Étape 2 : Le Cœur d'Inférence (BentoML)

Service Bento

Logique de Transformation

Étape 3 : La Passerelle Sécurisée (FastAPI)

Schémas Pydantic

Client Interne

Étape 4 : Fiabilité et Validation (Tests)

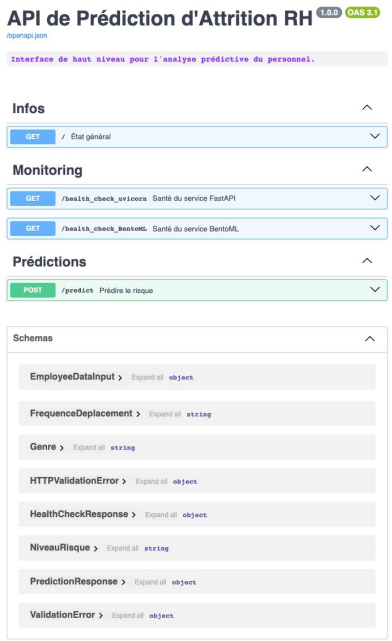
Tests de Validation (Human)

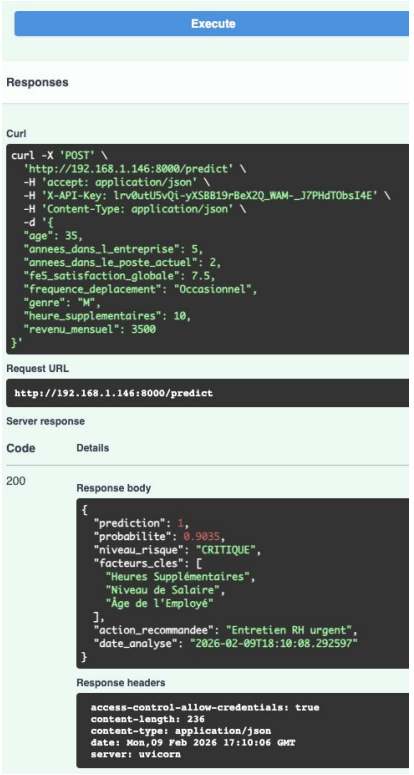
Tests de Validation (pytest)

Rapport de couverture de tests

Étape 4 : Fiabilité et Validation (Tests)

Tests de Validation (Human): Consultation d'attrition. FastAPI (Port 8000)





4 Conception Détaillée

ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

Modélisation SQL

Étape 2 : Le Cœur d'Inférence (BentoML)

Service Bento

Logique de Transformation

Étape 3 : La Passerelle Sécurisée (FastAPI)

Schémas Pydantic

Client Interne

Étape 4 : Fiabilité et Validation (Tests)

Tests de Validation (Human)

Tests de Validation (pytest)

Rapport de couverture de tests

Étape 4 : Fiabilité et Validation (Tests)

BentoML

TEST BENTO REQUEST

REQUEST

```
curl -X 'POST' \
  'http://192.168.1.146:3000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "employee_data": {
  "age": 35,
  "annees_dans_l_entreprise": 5,
  "annees_dans_le_poste_actuel": 2,
  "fe5_satisfaction_globale": 7.5,
  "frequence_deplacement": "Occasionnel",
  "genre": "M",
  "heure_supplementaires": 10,
  "revenu_mensuel": 3500
}
}
```

REPONSE

```
Response body
Download
{
  "prediction": 1,
  "probabilite": 0.9035,
  "niveau_risque": "CRITIQUE",
  "facteurs_cles": [
    "Heures Supplémentaires",
    "Niveau de Salaire",
    "Âge de l'Employé"
  ],
  "action_recommandee": "Entretien RH urgent",
  "date_analyse": "2026-02-09T16:28:03.159568"
}
```

FastAPI

TEST UNAUTHORIZED FAstAPI REQUEST

REQUEST

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "age": 35,
  "annees_dans_l_entreprise": 5,
  "annees_dans_le_poste_actuel": 2,
  "fe5_satisfaction_globale": 7.5,
  "frequence_deplacement": "Occasionnel",
  "genre": "M",
  "heure_supplementaires": 10,
  "revenu_mensuel": 3500
}'
```

REPONSE

```
Error: Unauthorized

Response body
Download
{
  "status": "error",
  "error": "Clé API manquante",
  "message": "L'en-tête 'X-API-Key' est requis.",
  "details": null,
  "timestamp": "2026-02-09T18:08:32.051958"
}
```

TEST AUTHORIZED FAstAPI REQUEST

REQUEST

```
curl -X 'POST' \
  'http://192.168.1.146:8000/predict' \
  -H 'accept: application/json' \
  -H 'X-API-Key: lrv0utU5vQi-XXX' \
  -H 'Content-Type: application/json' \
  -d '{
  "age": 35,
  "annees_dans_l_entreprise": 5,
  "annees_dans_le_poste_actuel": 2,
  "fe5_satisfaction_globale": 7.5,
  "frequence_deplacement": "Occasionnel",
  "genre": "M",
  "heure_supplementaires": 10,
  "revenu_mensuel": 3500
}'
```

REPONSE

```
{
  "prediction": 1,
  "probabilite": 0.9035,
  "niveau_risque": "CRITIQUE",
  "facteurs_cles": [
    "Heures Supplémentaires",
    "Niveau de Salaire",
    "Âge de l'Employé"
  ],
  "action_recommandee": "Entretien RH urgent",
  "date_analyse": "2026-02-09T18:10:08.292597"
}
```

4 Conception Détaillée

ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

Modélisation SQL

Étape 2 : Le Cœur d'Inférence (BentoML)

Service Bento

Logique de Transformation

Étape 3 : La Passerelle Sécurisée (FastAPI)

Schémas Pydantic

Client Interne

Étape 4 : Fiabilité et Validation (Tests)

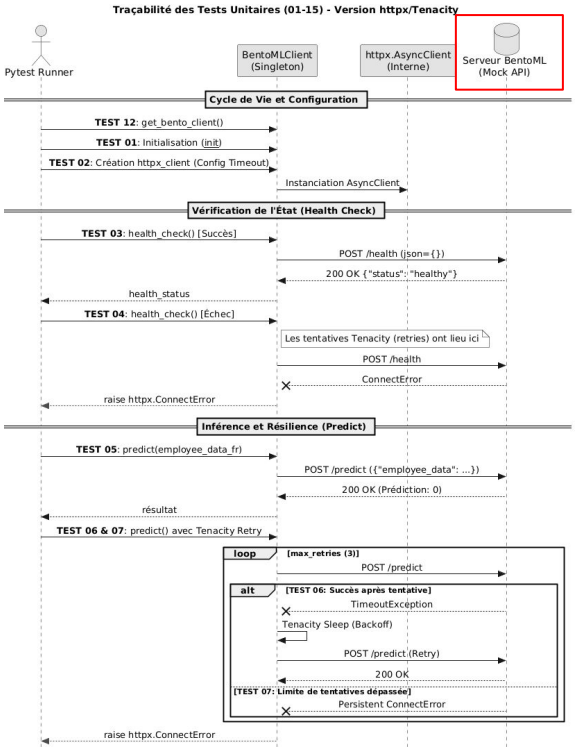
Tests de Validation (Human)

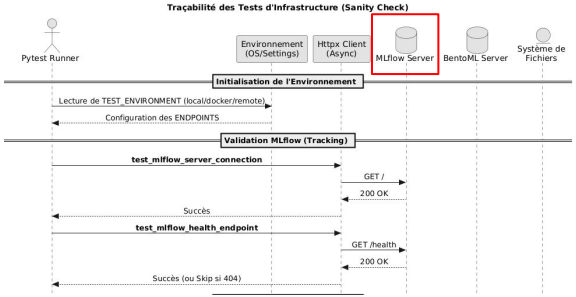
Tests de Validation (pytest)

Rapport de couverture de tests

Étape 4 : Fiabilité et Validation (Tests)

Tests de Validation (pytest)





Codes de Statut API
2xx (Succès) :
200 : OK (Tout va bien).
201 : Créé (Ressource enregistrée).
204 : Succès sans contenu (ex: après une suppression).

4xx (Erreurs Client) :
400 : Mauvaise requête (Syntaxe JSON cassée).
401/403 : Non autorisé / Interdit (Problème de Token).
404 : Introuvable (Mauvaise URL).
422 : Erreur de Validation (Format JSON correct, mais données invalides, ex: du texte au lieu d'un nombre).

5xx (Erreurs Serveur - Crash du code) :
500 : Erreur interne (Ton Python a planté).
503 : Service indisponible (BentoML ou serveur saturé).
504 : Timeout (Le modèle de ML est trop lent à répondre).

Chaque test est classifié selon sa nature:
- **Tests Unitaires** (Logique pure)
- **Tests d'Intégration** (Communication entre modules)
- **Tests de Bout en Bout / E2E**

```
@pytest.mark.integration
@pytest.mark.adapter_primary
@pytest.mark.security
@pytest.mark.fast
def test_predict_unauthorized():
```

4 Conception Détaillée

ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

Modélisation SQL

Étape 2 : Le Cœur d'Inférence (BentoML)

Service Bento

Logique de Transformation

Étape 3 : La Passerelle Sécurisée (FastAPI)

Schémas Pydantic

Client Interne

Étape 4 : Fiabilité et Validation (Tests)

Tests de Validation (Human)

Tests de Validation (pytest)

Rapport de couverture de tests

Étape 4 : Fiabilité et Validation (Tests)

Création de Tests de Validation (pytest)

1. Tests de Sécurité & Accès (Cross-Cutting Concerns)
Garantir l'intégrité de la porte d'entrée.
- Validation des accès
- Cryptographie
- Standardisation

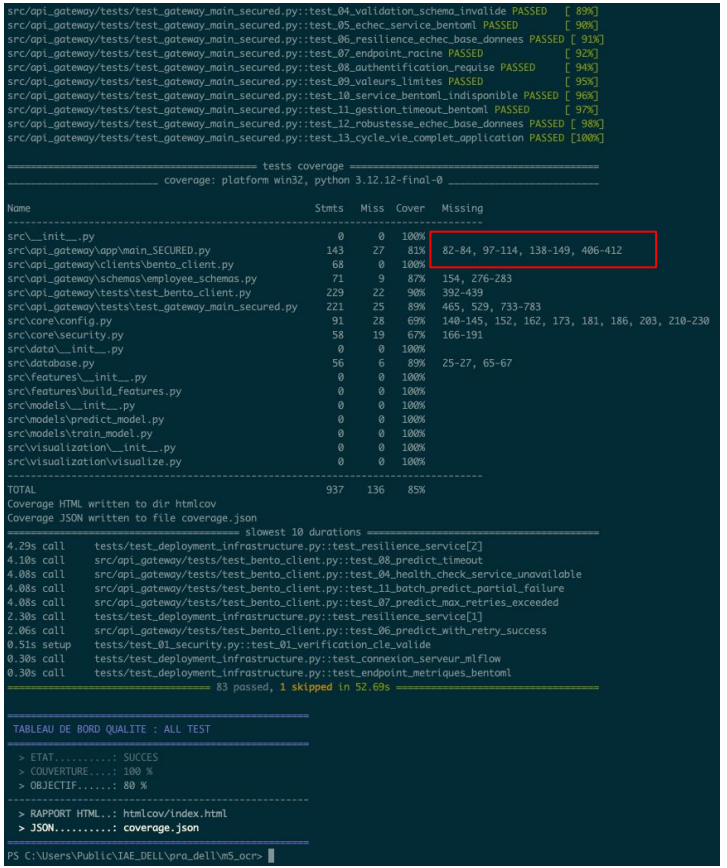
2. Tests de Validation & Schémas (Application Layer)
Filtrer les données corrompues avant qu'elles n'atteignent le modèle.
- Contraintes métier
- Intégrité structurelle
- Limites extrêmes

3. Tests de l'Adaptateur Secondaire (Persistance & Inférence)
Isoler et valider la communication avec les services externes.
- Base de Données :
- Client BentoML
- Intégration SQL

4. Tests de Résilience & Infrastructure (Performance & Robustesse)
Garantir la continuité de service en mode dégradé.
- Gestion des pannes
- SLAs & Performance
- Infrastructure

5. Tests d'Intégration Flux Complet (Primary Adapter)
Valider l'orchestration finale au niveau de l'API Gateway.
- Cycle de vie
- Flux nominal

--cov-report=term-missing



4 Conception Détaillée

ÉTAPE 2 : La Phase d'Inférence

Étape 1 : Fondations et Persistance (PostgreSQL)

Modélisation SQL

Étape 2 : Le Cœur d'Inférence (BentoML)

Service Bento

Logique de Transformation

Étape 3 : La Passerelle Sécurisée (FastAPI)

Schémas Pydantic

Client Interne

Étape 4 : Fiabilité et Validation (Tests)

Tests de Validation (Human)

Tests de Validation (pytest)

Rapport de couverture de tests

Étape 4 : Fiabilité et Validation (Tests)

Rapport de couverture de tests

Script d'Exécution (.ps1)	Fichier de Test (.py)	Couche Hexagonale	Type de Validation
`run_test_01_core_security.ps1`	`test_01_security.py`	**Cross-Cutting**	Sécurité, Hachage et Clés API
`run_test_03_database.ps1`	`test_03_database.py`	**Adapt. Secondaire**	Persistance SQL et Connexion DB
`run_test_05_bento_tests.ps1`	`test_bento_client.py`	**Adapt. Secondaire**	Client d'Inférence et Stratégies de Retry
`run_test_06_gateway_main_secured.ps1`	`test_gateway_main_secured.py`	**Adapt. Primaire**	Port HTTP et Cycle de Vie de l'API
`run_application_test_predict_logic.ps1`	`test_application_predict_logic.py`	**Application**	Logique Métier et Orchestration des flux
`run_deployment_test_infrastructure.ps1`	`test_deployment_infrastructure.py`	**Infrastructure**	Disponibilité (Readiness) MLflow et BentoML
`run_tests_ALL_with_coverage.ps1`	*(Tous les fichiers)*	**Global**	Rapport de Couverture de Code (100%)

Coverage report: 85%

Files | Functions | Classes

coverage.py v7.13.2, created at 2026-02-10 10:59 +0100

File ▲	statements	missing	excluded	coverage
src \ __init__.py	0	0	0	100%
src \ api_gateway \ app \ main_SECURED.py	143	27	0	81%
src \ api_gateway \ clients \ bento_client.py	68	0	0	100%
src \ api_gateway \ schemas \ employee_schemas.py	71	9	0	87%
src \ api_gateway \ tests \ test_bento_client.py	229	22	0	90%
src \ api_gateway \ tests \ test_gateway_main_secured.py	221	25	0	89%
src \ core \ config.py	91	28	0	69%
src \ core \ security.py	58	19	0	67%
src \ data \ __init__.py	0	0	0	100%
src \ database.py	56	6	0	89%
src \ features \ __init__.py	0	0	0	100%
src \ features \ build_features.py	0	0	0	100%
src \ models \ __init__.py	0	0	0	100%
src \ models \ predict_model.py	0	0	0	100%
src \ models \ train_model.py	0	0	0	100%
src \ visualization \ __init__.py	0	0	0	100%
src \ visualization \ visualize.py	0	0	0	100%
Total	**937**	**136**	**0**	**85%**

5 CI/CD

Chaîne de traitement d'Intégration et de Déploiement Continus)

1. L'Intégration Continue (CI - Continuous Integration)

2. Le Déploiement Continu (CD - Continuous Deployment)

L'inférence

Consommation de l'API BentoML

Générer le scoring prédictif de l'employé

CI/CD

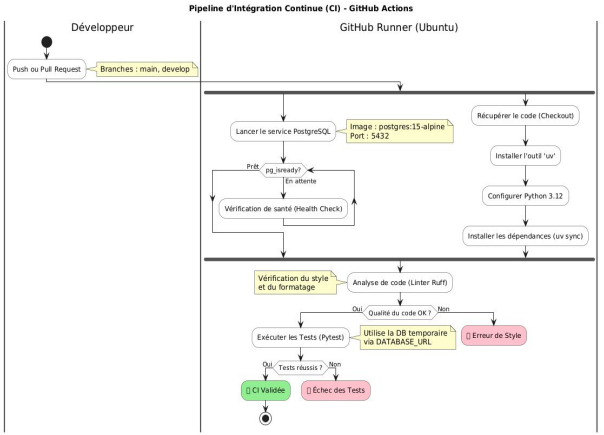
Implémentation de 2 Github workflows

Intégration Continue (ci.yml)

- **Pipeline d'Intégration Continue (CI) :** Chaque push ou Pull Request déclenche une batterie de tests automatisés sur un **Runner GitHub (Ubuntu)** pour garantir l'intégrité du code.

- **Validation en Environnement Isolé :** Le workflow lance un **conteneur PostgreSQL** temporaire pour valider les interactions avec la base de données et vérifie la santé du service (Health Check) avant toute autre étape.

- **Qualité et Conformité du Code :** Analyse statique automatique (Linting/Formatage **Ruff**) pour maintenir une dette technique minimale et assurer la lisibilité du projet par l'équipe.



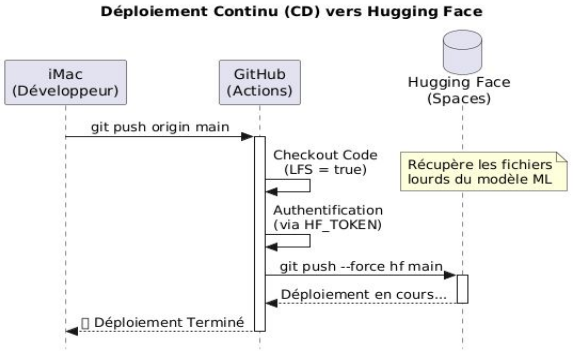
Interface Utilisateur pour Hugging Face Spaces

Creation d'une application **app.py** pour offrir une interface de prédiction interactive et accessible (**Gradio** éléments visuels: boutons, sliders,..).

Deployment Continu (deploy.yml)

- **Déploiement Continu (CD) vers Hugging Face :** Une fois les tests validés, GitHub Actions synchronise automatiquement le dépôt vers Hugging Face Spaces via **git push --force** , incluant la récupération des fichiers lourds (LFS) du modèle ML.

- **Sécurité des Secrets :** Utilisation de tokens d'authentification (HF_TOKEN) sécurisés dans les variables d'environnement de GitHub para protéger l'accès à l'infrastructure de production.



5 CI/CD

Chaîne de traitement d'Intégration et de Déploiement Continus)

1. L'Intégration Continue (CI - Continuous Integration)

2. Le Déploiement Continu (CD - Continuous Deployment)

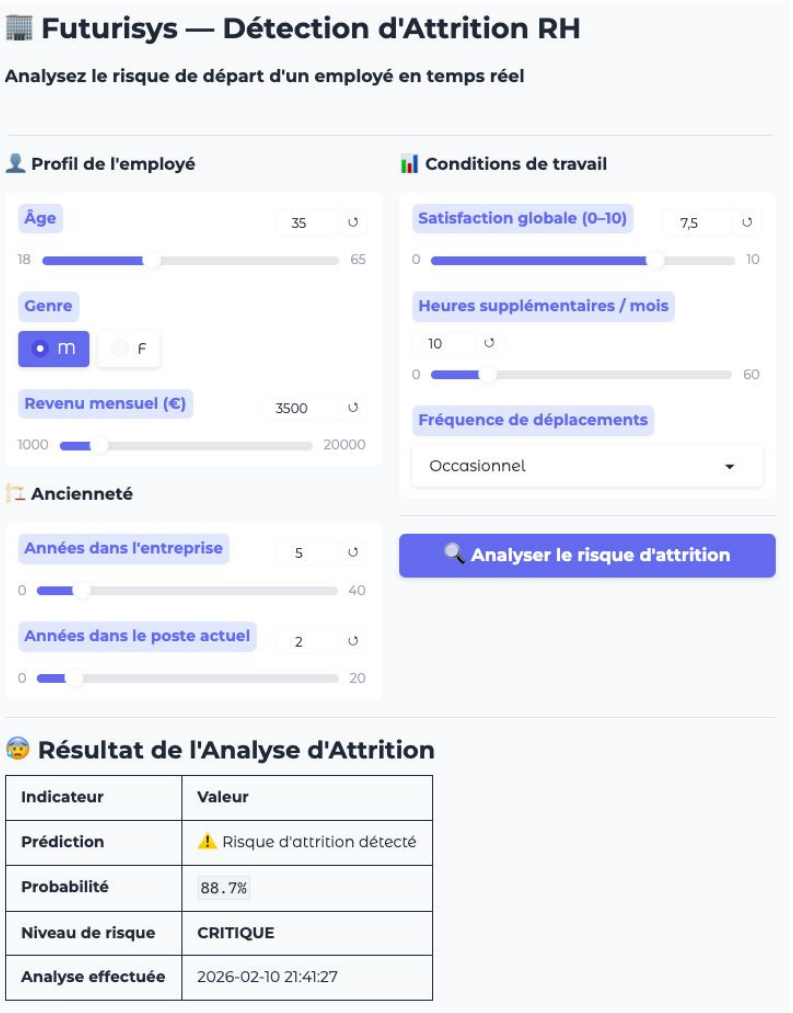
L'inférence

Consommation de l'API BentoML

Générer le scoring prédictif de l'employé

CI/CD

HuggingFaces Inference



Bilan du projet réalisé

💡 Observations Administratives

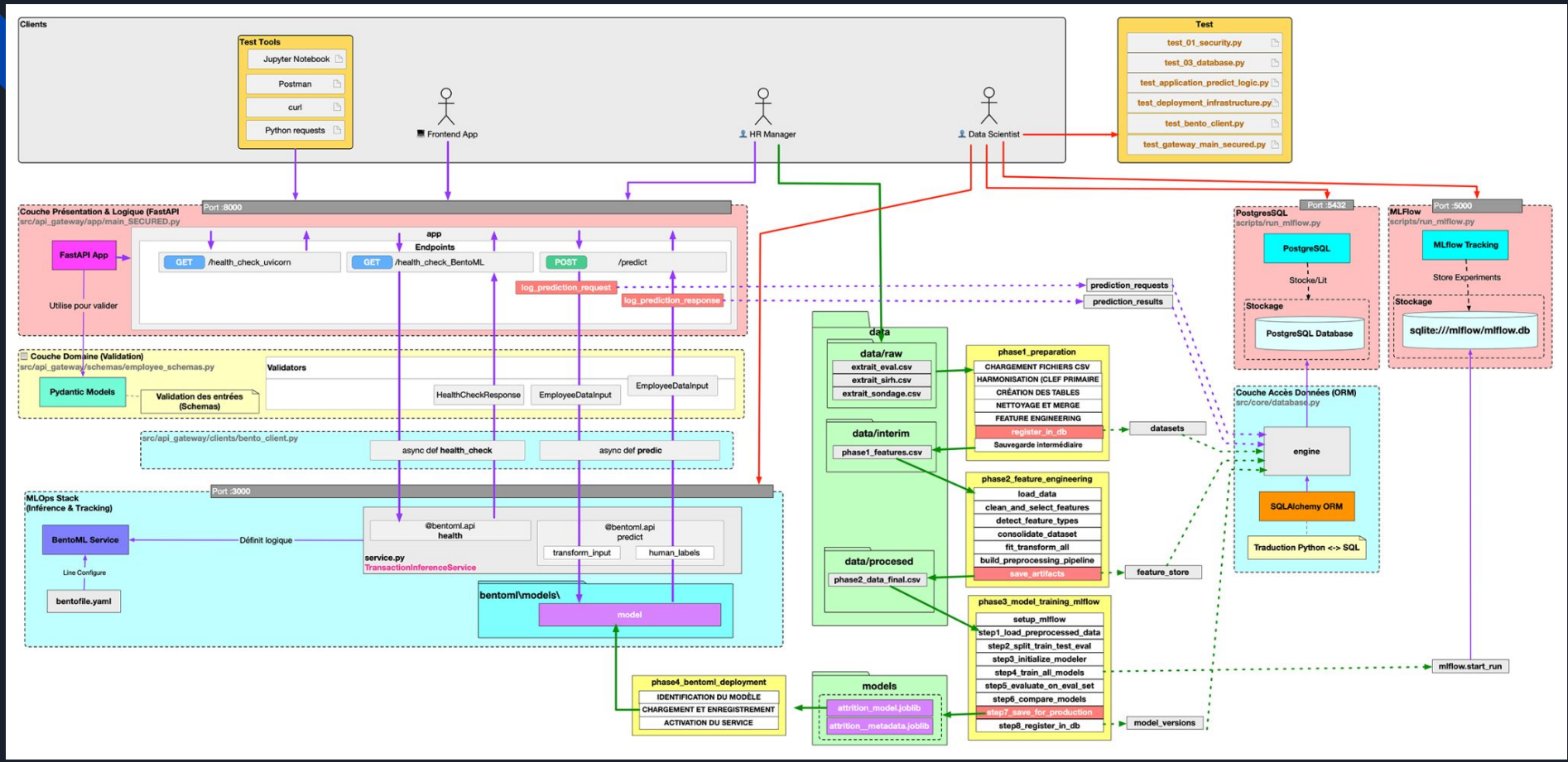
- Recevoir les données plus récentes pour commencer. Si nécessaire, le système est prêt pour ré-entraînements futures.
- Ensemble de Features réduit aux plus pertinents. Si nécessaire, capacité de réajuster avec un nouvelle Feature Engineering.
- Stratégie de deployment: "Recreate Deployment" sur Hugging Face Spaces (SaaS)

  Si nouveau modele, l'ancienne instance est arrêtée, le nouveau conteneur est construit (Build) puis lancé.

🚧Les défis techniques rencontrés

- La Logique de Transformation des Request/Response entre le Modèle et l'interface API avec un Human.
- Réalisation de couverture de Test supérieur à 80%
- Le déploiement du Modèle dans Hugging Faces. (Fichiers volumineux, PNGs considérés comme binaires, Interface nécessitant aussi de la Logique de Transformation,..)

Q & A


Clients
Test Tools
Jupyter Notebook
Postman
curl
Python requests
Frontend App
HR Manager
Data Scientist
Test
test_01_security.py
test_03_database.py
test_application_predict_logic.py
test_deployment_infrastructure.py
test_bento_client.py
test_gateway_main_secured.py
Couche Présentation & Logique (FastAPI
src/api_gateway/app/main_SECURED.py
Port :8000
FastAPI App
Utilise pour valider
app
Endpoints
GET /health_check_uvicorn
GET /health_check_BentoML
POST /predict
log_prediction_request
log_prediction_response
prediction_requests
prediction_results
PostgreSQL
scripts/run_mlflow.py
Port :5432
PostgreSQL
Stocke/Lit
Stockage
PostgreSQL Database
MLFlow
scripts/run_mlflow.py
Port :5000
MLflow Tracking
Store Experiments
Stockage
sqlite:///mlflow/mlflow.db
Couche Domaine (Validation)
src/api_gateway/schemas/employee_schemas.py
Pydantic Models
Validation des entrées (Schemas)
Validators
HealthCheckResponse
EmployeeDataInput
EmployeeDataInput
src/api_gateway/clients/bento_client.py
async def health_check
async def predic
data
data/raw
extrait_eval.csv
extrait_sirh.csv
extrait_sondage.csv
phase1_preparation
CHARGEMENT FICHIERS CSV
HARMONISATION (CLEF PRIMAIRE
CRÉATION DES TABLES
NETTOYAGE ET MERGE
FEATURE ENGINEERING
register_in_db
Sauvegarde intermédiaire
datasets
data/interim
phase1_features.csv
phase2_feature_engineering
load_data
clean_and_select_features
detect_feature_types
consolidate_dataset
fit_transform_all
build_preprocessing_pipeline
save_artifacts
feature_store
data/procesed
phase2_data_final.csv
Couche Accès Données (ORM)
src/core/database.py
engine
SQLAlchemy ORM
Traduction Python <-> SQL
Port :3000
MLOps Stack
(Inférence & Tracking)
BentoML Service
Définit logique
Line Configure
bentofile.yaml
@bentoml.api
health
@bentoml.api
predict
transform_input
human_labels
service.py
TransactionInferenceService
bentoml\models\
model
phase3_model_training_mlflow
setup_mlflow
step1_load_preprocessed_data
step2_split_train_test_eval
step3_initialize_modeler
step4_train_all_models
step5_evaluate_on_eval_set
step6_compare_models
step7_save_for_production
step8_register_in_db
model_versions
mlflow.start_run
phase4_bentoml_deployment
IDENTIFICATION DU MODÈLE
CHARGEMENT ET ENREGISTREMENT
ACTIVATION DU SERVICE
models
attrition_model.joblib
attrition__metadata.joblib